

# E-Learning JCB — Guion de Demostración Práctica (15 minutos)

Sigue a la exposición teórica de 15 min App desplegada: <https://e-learning-jcb-reflex-gray-orca.reflex.run/> Tener abierto el navegador antes de empezar · Credenciales preparadas

## ESTRUCTURA

| Demo | Funcionalidad                           | Tiempo |
|------|-----------------------------------------|--------|
| D1   | Página principal y navegación           | 1 min  |
| D2   | Registro e inicio de sesión             | 2 min  |
| D3   | Catálogo y búsqueda en tiempo real      | 2 min  |
| D4   | Visor de curso — experiencia estudiante | 2 min  |
| D5   | Dashboard instructor y CRUD de cursos   | 3 min  |
| D6   | Dashboard administrador                 | 2 min  |
| D7   | Protección de rutas por rol             | 1 min  |
| D8   | Conexión código Python → UI             | 2 min  |

## CREDENCIALES

| Rol        | Email                             | Contraseña    |
|------------|-----------------------------------|---------------|
| Estudiante | maria.garcia@elearningjcb.com     | student123    |
| Instructor | carlos.rodriguez@elearningjcb.com | instructor123 |
| Admin      | ana.martinez@elearningjcb.com     | admin123      |

# D1 — PÁGINA PRINCIPAL Y NAVEGACIÓN

---

🕒 1 minuto

---

*(Pantalla en / — página de inicio)*

Esto es lo que ve cualquier visitante. Los números que aparecen —cursos, instructores, estudiantes— son datos reales de MongoDB. No hay ningún número hardcodeado.

Fijaos en la **navbar**: sin sesión, solo muestra "Iniciar Sesión" y "Registrarse". En cuanto el usuario se autentique, el menú cambiará automáticamente sin recargar la página.

Navegamos al catálogo de cursos.

## D2 — REGISTRO E INICIO DE SESIÓN

---

🕒 2 minutos

---

(Navegar a </register>)

El formulario de registro tiene validación en Python. Si el email ya existe, el mensaje de error es genérico —"credenciales incorrectas"— nunca revela si el email está registrado o no. Eso es seguridad by design.

(Navegar a </login>, iniciar sesión como estudiante)

Observad lo que pasa: la navbar cambia **instantáneamente**. Aparece el menú de usuario con el nombre. Esto es el estado reactivo de Reflex: `AuthState.is_authenticated` pasa a `True` y todos los componentes que dependen de ese valor se actualizan solos vía WebSocket.

El sistema redirige automáticamente al dashboard de estudiante porque el rol detectado es `"student"`.

## D3 — CATÁLOGO Y BÚSQUEDA EN TIEMPO REAL

---

🕒 2 minutos

---

(Navegar a </courses>)

Los cursos se cargan desde MongoDB al montar la página.

(Escribir en el buscador)

Los resultados se filtran **mientras escribo**, sin hacer ninguna petición al servidor. Esto es la var computada `filtered_courses` que vimos en la exposición: se recalcula en Python cada vez que cambia `search_query`, Reflex sincroniza el resultado con la UI automáticamente.

Si borro el texto, vuelven todos los cursos. Cero latencia, cero llamadas al servidor.

Las categorías del filtro se gestionan desde el panel de administrador y se cargan dinámicamente.

## D4 — VISOR DE CURSO (EXPERIENCIA ESTUDIANTE)

---

🕒 2 minutos

---

*(Hacer clic en un curso)*

Página de detalle del curso. Si el estudiante no está inscrito, ve la información y el botón de inscripción.

*(Inscribirse o entrar a curso ya inscrito)*

Al estar inscrito, accedemos al **visor de curso**:

- **Sidebar izquierdo:** índice de lecciones. La lección actual está resaltada.
- **Panel central:** contenido de la lección activa.
- **Progreso:** cada lección visitada queda marcada como completada en MongoDB.

*(Navegar entre lecciones)*

Cambiar de lección actualiza el panel central instantáneamente. El progreso se guarda en base de datos — el estudiante puede retomarlo desde cualquier dispositivo.

## D5 — DASHBOARD INSTRUCTOR Y CRUD DE CURSOS

---

🕒 3 minutos

---

*(Cerrar sesión → iniciar como instructor)*

El sistema redirige a `/instructor/dashboard` automáticamente — mismo código de login, distinta redirección según el rol.

El dashboard muestra los cursos propios del instructor y sus KPIs: alumnos inscritos, ingresos, valoraciones.

*(Hacer clic en "Nuevo Curso")*

El formulario maneja tanto **creación como edición**. El flag `is_editing` distingue los dos modos: cuando es `False`, al guardar se crea un documento nuevo en MongoDB vinculado al instructor con `$addToSet` — garantiza no añadir duplicados de forma atómica.

*(Rellenar el formulario y guardar)*

El curso aparece en la lista inmediatamente.

*(Hacer clic en editar)*

El mismo formulario se reutiliza. `is_editing` es `True`, al guardar ejecuta `update_one` en lugar de `insert_one`.

*(Mostrar el dialog de confirmación de borrado)*

El **dialog de confirmación** antes de eliminar evita borrados accidentales. Componente de Reflex puro — sin JavaScript.

## D6 — DASHBOARD ADMINISTRADOR

---

🕒 2 minutos

---

*(Cerrar sesión → iniciar como administrador)*

El administrador llega a </admin/dashboard>. Las estadísticas globales se inyectan desde MongoDB en tiempo de carga: total de usuarios, cursos, inscripciones activas. Nada hardcodeado.

Desde aquí el administrador puede:

- **Gestionar usuarios:** ver todos los usuarios, cambiar roles, desactivar cuentas.
- **Gestionar cursos:** visibilidad global del catálogo, editar o eliminar cualquier curso.
- **Gestionar categorías:** añadir, editar o eliminar categorías del filtro público. Cualquier cambio se refleja inmediatamente en la página de catálogo.

Toda esta gestión en Python puro, sin ningún panel de administración externo.

## D7 — PROTECCIÓN DE RUTAS POR ROL

---

🕒 1 minuto

---

(Con sesión de estudiante, intentar navegar manualmente a `/admin/dashboard`)

Escribo directamente en la barra de direcciones `/admin/dashboard`.

El sistema muestra la **pantalla de acceso denegado** — no redirige, no crashea, renderiza el componente de error. Esto es `rx.cond(AuthState.is_user_admin, contenido_admin, pantalla_error)`.

(Intentar `/instructor/dashboard` con cuenta de estudiante)

Mismo resultado. La protección no es solo visual: los servicios de MongoDB tampoco ejecutan operaciones si el estado no tiene el rol correcto. **Doble capa de seguridad.**

# D8 — CONEXIÓN CÓDIGO PYTHON → UI

---

🕒 2 minutos

---

*(Mostrar fragmento de código junto al navegador)*

Para cerrar, quiero mostrar directamente la conexión entre una línea de Python y lo que acabáis de ver.

Esta es la navbar:

```
rx.cond(
    AuthState.is_authenticated,
    rx.menu.root(rx.menu.trigger(rx.text(AuthState.user_name)), ...),
    rx.hstack(rx.link("Iniciar Sesión"), rx.button("Registrarse"))
)
```

No hay HTML. No hay JavaScript. `rx.cond` en Python se compila a un componente React condicional. `AuthState.user_name` es una var reactiva que el framework sincroniza automáticamente.

Y este es el responsive design de la cuadrícula de cursos:

```
rx.grid(
    columns=rx.breakpoints(initial="1", sm="2", md="3", lg="4"),
    gap="4",
)
```

Una línea genera los cuatro breakpoints de CSS. Lo que habrían sido 20 líneas de media queries es una llamada a función en Python.

Eso resume el proyecto: **un lenguaje, una forma de pensar, una aplicación completa en producción.**

Muchas gracias. Quedo abierto a preguntas.

## APÉNDICE — RESPUESTAS A PREGUNTAS FRECUENTES

---

**¿Por qué autónomo y no Sociedad Limitada?** Para el primer año, el autónomo es más ágil y barato. La tarifa plana reduce la SS a 80€/mes. Si los beneficios superan 40.000€, el cambio a SL es más eficiente fiscalmente.

**¿Qué pasa con el RGPD y los datos de los estudiantes?** Los datos se almacenan en MongoDB Atlas con cifrado AES-256 en reposo y TLS en tránsito. La plataforma incluye política de privacidad, gestión de cookies y formularios con consentimiento explícito.

**¿No es más lento que usar React directamente?** En producción, Reflex compila a React optimizado. La diferencia de rendimiento es mínima para este tipo de aplicación. La ganancia en velocidad de desarrollo es enorme.

**¿Se puede usar SQL en lugar de MongoDB?** Sí. Reflex funciona con cualquier base de datos. MongoDB se eligió por la flexibilidad de esquema para cursos con estructura variable.

**¿Cómo se hace el deploy?** Reflex Cloud con un solo comando: `reflex deploy`. La app está en producción en: <https://e-learning-jcb-reflex-gray-orca.reflex.run/>

**¿Qué fue lo más difícil de desarrollar?** El sistema de estados con herencia y entender cómo el estado reactivo se sincroniza con el frontend. Requiere cambiar la forma de pensar respecto al desarrollo web tradicional.

---

*Guion de demostración práctica — 15 minutos — E-Learning JCB Sigue a: GUION\_EXPOSICION\_15MIN.md*